

算法详情

1. transformer

链接参考：

(粗略版)

<https://zhuanlan.zhihu.com/p/338817680>

□ 待看 (详细版)

<https://zhouyifan.net/2022/11/12/20220925-Transformer/>

softmax在绝对值较大的区域梯度较小，梯度下降的速度比较慢。如何理解这句话？

<https://www.kimi.com/share/d0rsq70onf4k74d328qg>

把query, key建模成向量，用向量相似度代替查询的判断语句，用加权和代替直接取值再求平均值。“注意力”，其实指的就是这里的权重。

算平均值时，权重的和应该是1。因此，我们可以用softmax把这些权重归一化一下，再算value的加权和。

Self-Attention 的计算过程

输入嵌入：假设输入序列的嵌入表示为 $X \in \mathbb{R}^{n \times d_{\text{model}}}$ ，其中 n 是序列长度， d_{model} 是嵌入维度。

线性变换：

查询矩阵 $Q = XW^Q$ ，其中 $W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$

键矩阵 $K = XW^K$ ，其中 $W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$

值矩阵 $V = XW^V$ ，其中 $W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$

通常情况下， $d_k = d_v = d_{\text{model}}$ ，即 W^Q 、 W^K 和 W^V 的维度与 d_{model} 一致。

点积注意力：

计算注意力分数 $A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$

计算输出 Attention(Q, K, V) = AV

维度一致性

Q 和 K 的维度： $Q \in \mathbb{R}^{n \times d_k}$ 和 $K \in \mathbb{R}^{n \times d_k}$ ，因此 $QK^T \in \mathbb{R}^{n \times n}$ ，即每个位置的查询与所有位置的键进行点积，得到一个 $n \times n$ 的注意力分数矩阵。

V 的维度： $V \in \mathbb{R}^{n \times d_v}$ ，因此 $AV \in \mathbb{R}^{n \times d_v}$ ，即每个位置的输出是注意力权重与值的加权和。

总结

在 Self-Attention 机制中，Q 和 K 的维度必须一致，以确保可以进行点积运算。通常情况下， $d_k = d_v = d_{\text{model}}$ ，即 Q、K 和 V 的维度与输入嵌入的维度一致。这样设计可以确保 Self-Attention 机制能够有效地捕捉输入序列中的长距离依赖关系。

softmax在绝对值较大的区域梯度较小，梯度下降的速度比较慢。因此，我们要让被softmax的点乘数值尽可能小。

为什么要用一个和 d_k 成比例的项来放缩 QK^T 呢？这是因为，softmax在绝对值较大的区域梯度较小，梯度下降的速度比较慢。因此，我们要让被softmax的点乘数值尽可能小。而一般在 d_k 较大时，也就是向量较长时，点乘的数值会比较大。除以一个和 d_k 相关的量能够防止点乘的值过大。

相似度并不只有求点乘这一种方式。另一种常用的注意力函数叫做加性注意力，它用一个单层神经网络来计算两个向量的相似度。相比之下，点乘注意力算起来快一些。出于性能上的考量，论文使用了点乘注意力。

自注意力模块的目的是为每一个输入token生成一个向量表示，该表示不仅能反映token本身的性质，还能反映token在句子里特有的性质。

注意力的一种应用：通过让一句话中的每个单词去向其他单词查询信息，我们能为每一个单词生成一个更有意义的向量表示。

query, key, value不应该是人工指定的，而应该是可学习的。因此，我们可以用可学习的参数来描述从词嵌入到query, key, value的变换过程。

<https://zhuanlan.zhihu.com/p/648050614>

<https://developer.huawei.com/consumer/cn/forum/topic/0201152893086519250>

2. 稀疏算法

a. TF-IDF

(1) TF是词频(Term Frequency)

词频 (TF) 表示词条（关键字）在文本中出现的频率。

这个数字通常会被归一化(一般是词频除以文章总词数), 以防止它偏向长的文件。

公式：

$$tf_{ij} = \frac{n_{i,j}}{\sum_k n_{k,j}}$$

即：

$$TF_w = \frac{\text{在某一类中词条}w\text{出现的次数}}{\text{该类中所有的词条数目}}$$

其中 $n_{i,j}$ 是该词在文件 d_j 中出现的次数，分母则是文件 d_j 中所有词汇出现的次数总和

(2) IDF是逆向文件频率(Inverse Document Frequency)

逆向文件频率 (IDF)：某一特定词语的IDF，可以由总文件数目除以包含该词语的文件的数目，再将得到的商取对数得到。

如果包含词条t的文档越少，IDF越大，则说明词条具有很好的类别区分能力。

公式：

$$idf_i = \log \frac{|D|}{|\{j: t_i \in d_j\}|}$$

其中， $|D|$ 是语料库中的文件总数， $|\{j: t_i \in d_j\}|$ 表示包含词语 t_i 的文件数目 (即 $n_{i,j} \neq 0$ 的文件数目)。如果该词语不在语料库中，就会导致分母为零，因此一般情况下使用 $1 + |\{j: t_i \in d_j\}|$

$$IDF = \log\left(\frac{\text{语料库的文档总数}}{\text{包含词条}w\text{的文档数} + 1}\right), \text{分母之所以要加1, 是为了避免分母为0}$$

最终公式：

$$TF - IDF = TF * IDF$$

链接：

https://blog.csdn.net/asiale_bird/article/details/81486700

b. BM25算法

有一个query和一批文档Ds，现在要计算query和每篇文档D之间的相关性分数，我们的做法是，先对query进行切分，得到单词qi，然后单词的分数由3部分组成：

- query中每个单词qi和文档d之间的相关性
- 单词和query之间的相关性
- 每个单词的权重

最后对于每个单词的分数做一个求和，就得到了query和文档之间的分数。

链接：

<https://blog.csdn.net/laobai1015/article/details/120143102>

c. 倒排算法

倒排索引(Inverted Index)：倒排索引是实现“单词-文档矩阵”的一种具体存储形式，通过倒排索引，可以根据单词快速获取包含这个单词的文档列表。倒排索引主要由两个部分组成：“**单词词典**”和“**倒排文件**”。

- **单词词典(Lexicon)**：搜索引擎的通常索引单位是单词，单词词典是由文档集中出现过的所有单词构成的字符串集合，单词词典内每条索引项记载单词本身的一些信息以及指向“倒排列表”的指针。
- **倒排列表(PostingList)**：倒排列表记载了出现过某个单词的所有文档的文档列表及单词在该文档中出现的位置信息，每条记录称为一个倒排项(Posting)。根据倒排列表，即可获知哪些文档包含某个单词。
- **倒排文件(Inverted File)**：所有单词的倒排列表往往顺序地存储在磁盘的某个文件里，这个文件即被称之为倒排文件，倒排文件是存储倒排索引的物理文件。

1. 倒排索引：

单词ID(WordID)	单词(Word)	倒排列表(DocID;TF;<Pos>)
1	乔布斯	(1;1;<1>),(3;1;<6>),(4;1;<1>),(5;1;<1>)
2	苹果	(2;1;<1>),(3;1;<1>),(5;1;<5>)
3	iPad2	(3;1;<8>),(4;1;<7>)
4	宣布	(3;1;<7>)
5	了	(1;1;<3>),(4;1;<3>),(5;1;<3>)
...	...	...

TF(term frequency): 单词在文档中出现的次数。
Pos: 单词在文档中出现的位置。

文档ID， 单词频次， <单词位置>